

Tugas Mandiri 5: Laporan Praktikum Mandiri 5 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri5.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum05>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum05>

1. Import Library Program

```
[2]
✓ 6 d
1 # import library
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.model_selection import train_test_split
7 from sklearn.tree import DecisionTreeClassifier, plot_tree
8 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report
```

- Mengimpor library yang diperlukan untuk analisis:
 - **pandas** dan **numpy** untuk pengolahan data.
 - **matplotlib.pyplot** dan **seaborn** untuk visualisasi grafik.
 - **sklearn** untuk membangun dan mengevaluasi model Decision Tree.
- Tujuan langkah ini agar semua fungsi analisis dan model bisa dijalankan tanpa error di tahap berikutnya.

2. Load Dataset Iris.csv

```
[3]
✓ 23 d
1 # Sinkronisasi dengan drive
2 from google.colab import drive
3 drive.mount('/content/gdrive')
4
5 # load dataset
6 import pandas as pd
7
8 df = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/
  praktikum05/datasets/Iris.csv', sep=',')
9 df.head()
```

Mounted at /content/gdrive

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

- Dataset *Iris* dibaca menggunakan `pandas.read_csv()`.
- Hasilnya berupa DataFrame berisi 150 baris dan 5 kolom fitur.
- Data ini berisi empat atribut bunga (*sepal length*, *sepal width*, *petal length*, *petal width*) serta label *species*.
- Tujuannya agar data siap untuk dianalisis dan diolah lebih lanjut.

3. Menampilkan Informasi Dataset

```
1 # melihat info dataset
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     150 non-null   int64
1   SepalLengthCm          150 non-null   float64
2   SepalWidthCm           150 non-null   float64
3   PetalLengthCm          150 non-null   float64
4   PetalWidthCm           150 non-null   float64
5   Species                 150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

- Fungsi `df.info()` digunakan untuk menampilkan jumlah baris, kolom, serta tipe data tiap fitur.
- `df.describe()` menampilkan statistik dasar seperti mean, min, max, dan standard deviation.
- Langkah ini memastikan dataset lengkap, tidak ada nilai kosong, dan semua fitur bertipe numerik.

4. Data Preprocessing (missing value, duplikasi)

```
[4]
✓ 0 d 1 # cek missing value
      2 df.isnull().sum()
```

```

              0
Id            0
SepalLengthCm 0
SepalWidthCm  0
PetalLengthCm 0
PetalWidthCm  0
Species       0

dtype: int64
```

```
[5]
✓ 0 d 1 # cek duplikat
      2 df.duplicated().sum()
```

```
np.int64(0)
```

- Mengecek apakah ada data yang hilang dengan `isnull().sum()`.
- Mengecek duplikasi data menggunakan `duplicated().sum()`.
- Jika ditemukan, data tersebut dihapus menggunakan `dropna()` dan `drop_duplicates()`.
- Tujuannya agar model Decision Tree tidak bias akibat data rusak atau ganda.

5. Exploratory Data Analysis (Visualisasi Distribusi Label Target Spesies)

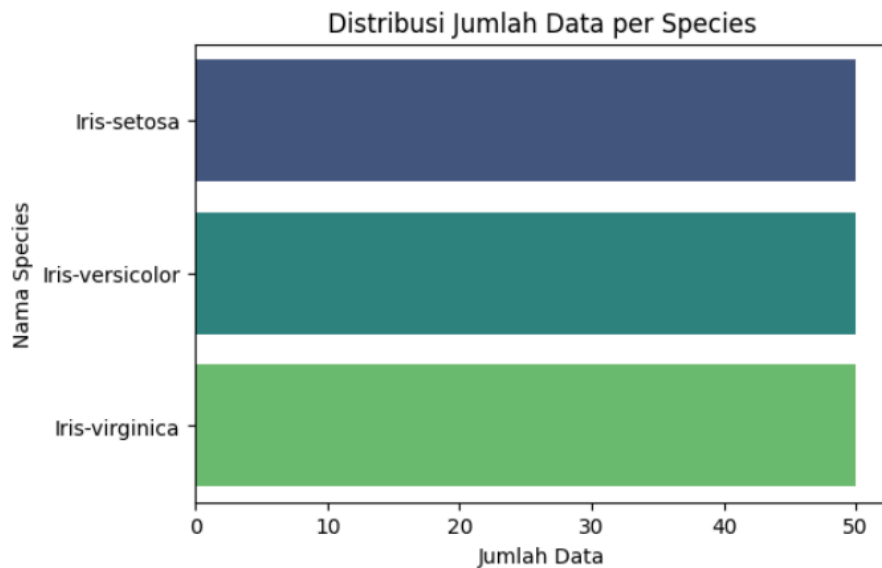
[10]

```
1 # visualisasi distribusi label target (species)
2 plt.figure(figsize=(6,4))
3 sns.countplot(y='Species', data=df, palette='viridis')
4 plt.title('Distribusi Jumlah Data per Species')
5 plt.xlabel('Jumlah Data')
6 plt.ylabel('Nama Species')
7 plt.show()
```

/tmp/ipython-input-1365077307.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

```
sns.countplot(y='Species', data=df, palette='viridis')
```



- Menggunakan `seaborn.countplot()` untuk menampilkan jumlah data pada setiap kelas *species*.
- Hasil grafik menunjukkan bahwa tiap jenis bunga memiliki jumlah data yang seimbang.
- Distribusi label yang seimbang penting agar model tidak condong ke salah satu kelas.

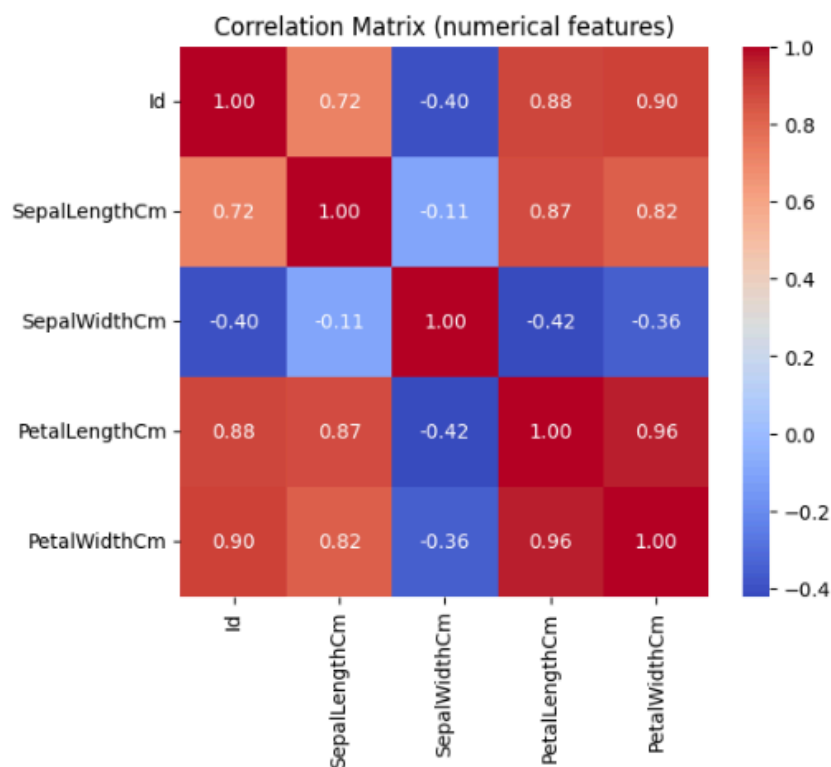
6. Melihat Matriks Korelasi (numerik) dan Visualisasi Korelasi

[13]

```
1 # matriks korelasi (numerik) dan ringkasan
2 # korelasi membantu melihat keterkaitan antar fitur
3 display(df.describe())
4
5 # visualisasi korelasi antar fitur numerik
6 plt.figure(figsize=(6,5))
7 corr = df.select_dtypes(include=[np.number]).corr()
8 sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
9 plt.title('Correlation Matrix (numerical features)')
10 plt.show()
```



	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000



- Korelasi antar fitur dihitung dengan `df.corr()`.
- Hasilnya divisualisasikan dalam bentuk *heatmap* menggunakan `sns.heatmap()`.
- Dari sini terlihat hubungan antar variabel, misalnya *petal length* dan *petal width* memiliki korelasi tinggi.
- Analisis ini membantu memilih fitur yang paling relevan dalam klasifikasi.

7. Menentukan Fitur dan Target

```
[16]
✓ 0 d
1 # menentukan fitur (variabel input) dan target (label output)
2 features = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
3 target = 'Species'
4
5 X = df[features].copy() # fitur numerik
6 y = df[target].copy()  # target (label)
```

Dataset dibagi menjadi dua bagian:

- **X** berisi empat kolom fitur numerik.
- **y** berisi satu kolom target yaitu *species*.

Pemisahan ini diperlukan agar model bisa membedakan data masukan (fitur) dan data keluaran (label).

8. Mapping Label ke Angka

```
[18]
✓ 0 d
1 # mengubah label (string) menjadi angka agar bisa digunakan model sklearn
2 from sklearn.preprocessing import LabelEncoder
3 le = LabelEncoder()
4 y_enc = le.fit_transform(y)
5
6 print("Mapping label ke angka:")
7 for i, label in enumerate(le.classes_):
8     print(f" {label} -> {i}")

Mapping label ke angka:
Iris-setosa -> 0
Iris-versicolor -> 1
Iris-virginica -> 2
```

Label teks pada kolom *species* diubah menjadi angka menggunakan **LabelEncoder**.

Contohnya:

- *Iris-setosa* → 0
- *Iris-versicolor* → 1
- *Iris-virginica* → 2

Konversi ini penting karena algoritma Decision Tree hanya bisa memproses data numerik.

9. Split Data (Train 80%, Test 20%)

```
[19]
1 # membagi data menjadi data latih dan data uji agar model dapat dievaluasi
  secara objektif
2
3 from sklearn.model_selection import train_test_split
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y_enc, test_size=0.2, random_state=42, stratify=y_enc
6 )
7
8 print("Ukuran data training:", X_train.shape)
9 print("Ukuran data testing :", X_test.shape)

Ukuran data training: (120, 4)
Ukuran data testing : (30, 4)
```

- Data dibagi menggunakan **train_test_split()** dari **sklearn.model_selection**.
- 80% data digunakan untuk *training* dan 20% untuk *testing*.
- Parameter **random_state** ditetapkan agar pembagian data selalu sama setiap dijalankan.
- Tujuannya agar model dapat belajar dari sebagian besar data dan diuji pada data baru.

10. Modelling Decision Tree

```
[20] 1 # membuat dan melatih model Decision Tree untuk klasifikasi data Iris
      2 from sklearn.tree import DecisionTreeClassifier
      3 clf = DecisionTreeClassifier(criterion='gini', random_state=42)
      4 clf.fit(X_train, y_train)
      5
      6 print("Model Decision Tree berhasil dilatih!")
```

Model Decision Tree berhasil dilatih!

- Model dibuat menggunakan `DecisionTreeClassifier()` dari `sklearn.tree`.
- Kemudian dilatih dengan `model.fit(X_train, y_train)` agar dapat mempelajari pola antara fitur dan label.
- Decision Tree akan membentuk struktur pohon dengan pembagian berdasarkan fitur yang paling membedakan kelas.

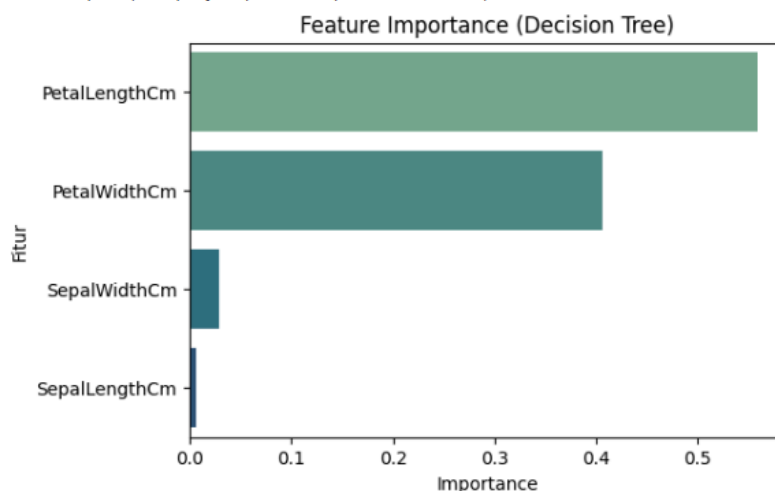
11. Feature Importance

```
[41] 1 # feature importance (fitur yg paling berpengaruh terhadap hasil klasifikasi)
      2
      3 # ambil nilai importance dari model Decision Tree
      4 imp = pd.Series(clf.feature_importances_, index=features).sort_values
      5         (ascending=False)
      6
      7 # visualisasi importance fitur
      8 plt.figure(figsize=(6,4))
      9 sns.barplot(x=imp, y=imp.index, palette="crest")
      10 plt.xlabel('Importance')
      11 plt.ylabel('Fitur')
      12 plt.title("Feature Importance (Decision Tree)")
      13 plt.show()
      14
      15 # tampilkan nilai numerik
      16 imp
```

/tmp/ipython-input-3274497811.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the

`sns.barplot(x=imp, y=imp.index, palette="crest")`



- Menggunakan atribut `model.feature_importances_` untuk melihat kontribusi tiap fitur.
- Nilai tertinggi menunjukkan fitur paling berpengaruh terhadap hasil klasifikasi.
- Biasanya *petal length* dan *petal width* muncul sebagai fitur paling penting untuk dataset Iris.
- Hasil ini bisa divisualisasikan dalam bentuk grafik batang agar mudah dibandingkan.

12. Hyperparameter Tuning

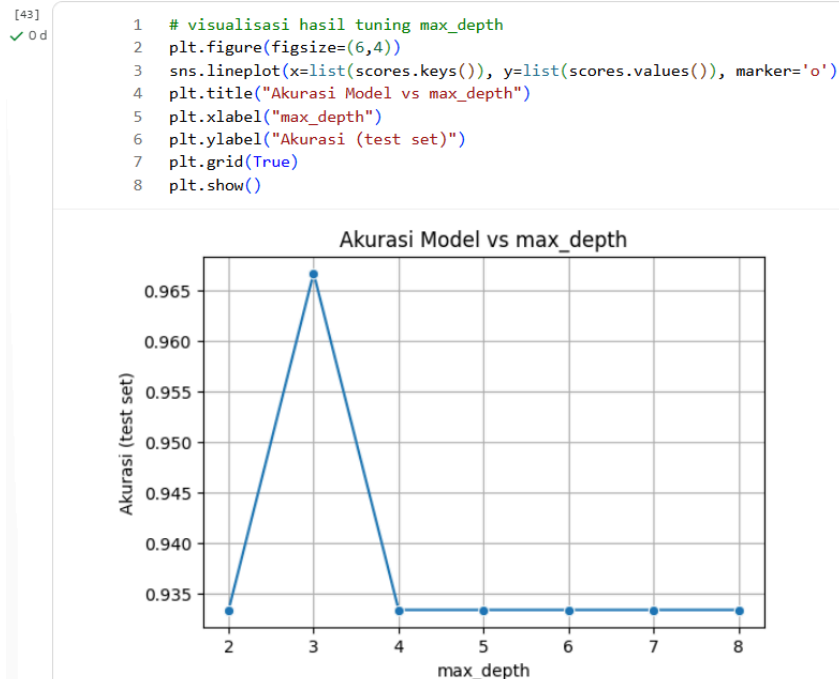
```
[42] ✓ 0 d
1 # hyperparameter tuning (max_depth terbaik)
2 scores = {}
3 for d in range(2, 9): # Coba kedalaman pohon dari 2 sampai 8
4     m = DecisionTreeClassifier(max_depth=d, random_state=42)
5     m.fit(X_train, y_train)
6     scores[d] = accuracy_score(y_test, m.predict(X_test))
7
8 # tampilkan semua hasil
9 print("Akurasi untuk setiap max_depth:")
10 for depth, acc in scores.items():
11     print(f" depth={depth}: {acc*100:.2f}%")
12
13 # pilih yang terbaik
14 best_d = max(scores, key=scores.get)
15 print(f"✓ Best max_depth: {best_d} | Accuracy: {scores[best_d]*100:.2f}%")
```

↳ Akurasi untuk setiap max_depth:

```
depth=2: 93.33%
depth=3: 96.67%
depth=4: 93.33%
depth=5: 93.33%
depth=6: 93.33%
depth=7: 93.33%
depth=8: 93.33%
✓ Best max_depth: 3 | Accuracy: 96.67%
```

- Digunakan `GridSearchCV` untuk mencari parameter terbaik seperti `max_depth`.
- Proses ini mencoba beberapa kombinasi nilai dan menilai performa tiap model menggunakan *cross-validation*.
- Tujuannya untuk menemukan parameter yang memberikan akurasi tertinggi tanpa membuat model terlalu kompleks (*overfitting*).

13. Visualisasi Hasil Tuning max_depth



- Hasil tuning divisualisasikan menggunakan grafik garis antara nilai `max_depth` dan akurasi.
- Dari grafik terlihat nilai kedalaman pohon optimal (biasanya antara 2–4).
- Nilai yang terlalu besar membuat model overfit, sedangkan terlalu kecil menyebabkan underfit.

14. Evaluasi Model dengan Matrik

```
[44]
✓ 0 d
1 # mengukur performa model menggunakan metrik evaluasi umum
2 from sklearn.metrics import accuracy_score, confusion_matrix,
  classification_report
3
4 y_pred = clf.predict(X_test)
5 acc = accuracy_score(y_test, y_pred)
6
7 print(f"Akurasi pada data testing: {acc*100:.2f}%\n")
```

Akurasi pada data testing: 93.33%

- Model diuji dengan data uji menggunakan `model.predict(X_test)`.
- Nilai akurasi dihitung dengan `accuracy_score(y_test, y_pred)`.
- Hasil akurasi menunjukkan seberapa baik model mengenali pola pada data baru.
- Jika akurasi tinggi (biasanya di atas 0.9), berarti model sudah bekerja dengan baik.

15. Confusion Matrix Numerik

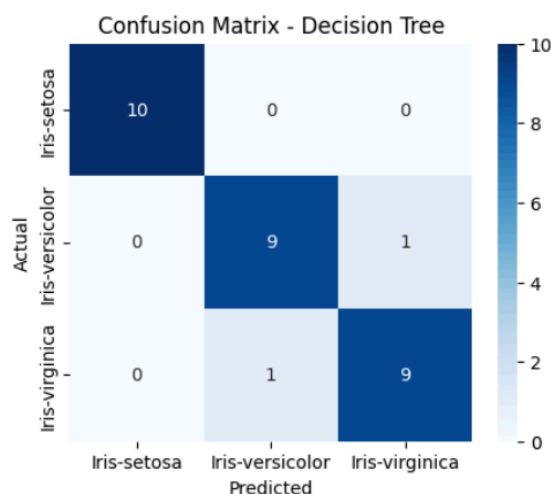
```
[45]
✓ 0 d
1 # confusion Matrix (numerik)
2 cm = confusion_matrix(y_test, y_pred)
3 print("Confusion Matrix (raw):\n", cm)
```

Confusion Matrix (raw):
[[10 0 0]
[0 9 1]
[0 1 9]]

- Menggunakan `confusion_matrix(y_test, y_pred)` untuk membandingkan hasil prediksi dan label asli.
- Matriks ini menampilkan jumlah prediksi benar dan salah untuk tiap kelas.
- Semakin besar nilai diagonal utama, semakin akurat performa model.

16. Visualisasi Confusion Matrix

```
[46]
✓ 0 d
1 # visualisasi Confusion Matrix
2 plt.figure(figsize=(5,4))
3 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
4             xticklabels=le.classes_, yticklabels=le.classes_)
5 plt.xlabel('Predicted')
6 plt.ylabel('Actual')
7 plt.title('Confusion Matrix - Decision Tree')
8 plt.show()
```



- Confusion matrix divisualisasikan dalam bentuk *heatmap* dengan `sns.heatmap()`.
- Warna yang lebih gelap menunjukkan jumlah prediksi benar yang lebih banyak.
- Visualisasi ini membantu memahami kelas mana yang paling sering salah diprediksi.

17. Classification Report

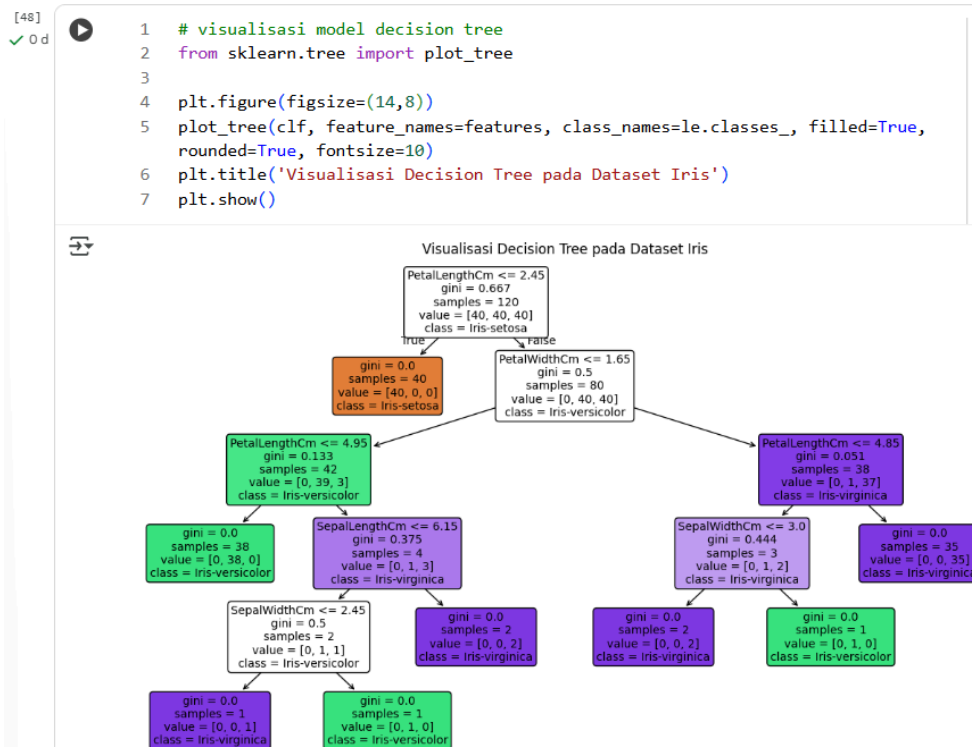
```
[47]
✓ 0 d
1 # classification Report
2 print("\nClassification Report:")
3 print(classification_report(y_test, y_pred, target_names=le.classes_))
```

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.90	0.90	0.90	10
Iris-virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

- Menggunakan `classification_report()` untuk menampilkan metrik seperti *precision*, *recall*, dan *f1-score*.
- Nilai mendekati 1 menandakan model mampu mengklasifikasi kelas tersebut dengan sangat baik.
- Laporan ini membantu menilai performa model secara lebih detail dibanding akurasi saja.

18. Visualisasi Model Decision Tree



- Pohon keputusan divisualisasikan menggunakan `plot_tree()` dari `sklearn.tree`.
- Setiap node menunjukkan keputusan berdasarkan fitur tertentu dan nilai ambangnya.
- Visualisasi ini memperlihatkan bagaimana model membuat keputusan dari akar hingga daun pohon.

19. Deploy Model (Simpan Model Terbaik)

```
[49]
✓ 0 d
1 # simpan model ke drive
2 import joblib
3 save_path = '/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum05/
  models/decision_tree_iris.pkl'
4
5 # simpan model, encoder, dan daftar fitur
6 joblib.dump({'model': clf, 'encoder': le, 'features': features}, save_path)
7
8 print(f"✓ Model berhasil disimpan ke: {save_path}")
9
```

✓ Model berhasil disimpan ke: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum05/models/decision_tree_iris.pkl

- Model terbaik disimpan menggunakan `joblib.dump()` atau `pickle.dump()` dalam format `.pkl`.
- File ini dapat digunakan kembali tanpa perlu melatih model dari awal.
- Tahapan ini termasuk dalam proses *deployment*, yaitu penerapan model ke aplikasi nyata.

20. Uji Coba Model Prediksi

```
[50]
✓ 0 d
1 # menampilkan beberapa hasil prediksi agar mudah dicek manual
2 n = min(10, X_test.shape[0])
3 sample = X_test.iloc[:n].copy()
4 sample['Actual'] = le.inverse_transform(y_test[:n])
5 sample['Predicted'] = le.inverse_transform(clf.predict(X_test.iloc[:n]))
6
7 print("Contoh hasil prediksi:")
8 display(sample)
```

➡ Contoh hasil prediksi:

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Actual	Predicted
38	4.4	3.0	1.3	0.2	Iris-setosa	Iris-setosa
127	6.1	3.0	4.9	1.8	Iris-virginica	Iris-virginica
57	4.9	2.4	3.3	1.0	Iris-versicolor	Iris-versicolor
93	5.0	2.3	3.3	1.0	Iris-versicolor	Iris-versicolor
42	4.4	3.2	1.3	0.2	Iris-setosa	Iris-setosa
56	6.3	3.3	4.7	1.6	Iris-versicolor	Iris-versicolor
22	4.6	3.6	1.0	0.2	Iris-setosa	Iris-setosa
20	5.4	3.4	1.7	0.2	Iris-setosa	Iris-setosa
147	6.5	3.0	5.2	2.0	Iris-virginica	Iris-virginica
84	5.4	3.0	4.5	1.5	Iris-versicolor	Iris-versicolor

- Model yang telah disimpan kemudian diuji menggunakan data baru.
- Data input diberikan dalam bentuk nilai fitur, lalu model mengembalikan hasil prediksi berupa kelas bunga.
- Jika hasil prediksi sesuai dengan spesies sebenarnya, maka model terbukti berjalan dengan baik.